

0.65.3. La fonction `defile()` — filer sur sa ligne sans arrêter le jeu

`#include <defile>` : `defile(numero, x, y, tuile, sens, vitesse)`, appelée à chaque image, fait filer une tuile sur sa ligne, d'un bord à l'autre, sans fin.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `defile(...)` fait **filer** une tuile sur sa ligne, sans bloquer : un pas quand c'est l'heure, et au bord elle **repart de l'autre côté**.

La ligne à écrire : `#include <defile>`. À ne pas confondre avec `defiler()`, qui fait glisser tout le fond.

Ses arguments : un numéro (0 à 3), le départ, la tuile, le sens (1 : à droite, -1 : à gauche), la vitesse en ms.

Ce qu'elle coûte : environ **300 octets**, plus la police entière.

Essaie : le sens -1.

On retrouve `defile` dans 4 leçons

[0.65.4](#), [0.76.13](#), [0.76.14](#), [0.90.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1141 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage – la mise en route de la console
defile()	237 o	← main() – une fonction fournie par le langage
main()	33 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles – la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 2	26 o sur place, dans defile() + AdresseCase (10 o)
effacer()	× 1	15 o sur place, dans defile() + AdresseCase (10 o), EffacerCases (12 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans defile()

Tes fonctions appelées :
defile() x 1 26 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

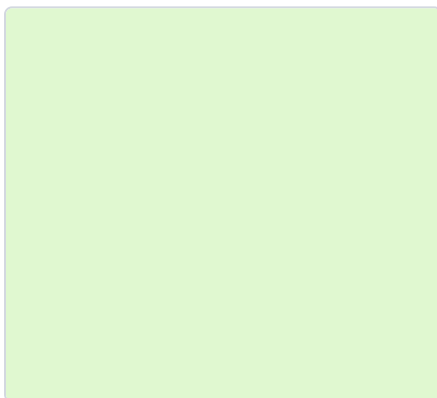
```
// ---- #include <defile> : filer sur sa ligne sans arrêter le jeu ----  
  
#include <defile>      // defile() : une tuile qui file, sans bloquer  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    while (true) {  
        image();  
        defile(0, 15, 8, ALPHABET[0], 1, 50);    // vers la droite, 50 ms par pas  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

Un A qui file vers la droite, et revient par la gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A atteint le bord droit
- ✓ et repart de la colonne 0

Ensuite : **0.65.4. Une lettre qui avance — en simple** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++